Publication-anchored paired snapshots of Python research software with field-level change records

Yanbo Bian[1,*] and Zhihai Wang[1,*]

[1]Beijing Jiaotong University, Beijing 100044, China, 24722081@bjtu.edu.cn; zhhwang@bjtu.edu.cn
[*]Correspondence to Yanbo Bian or Zhihai Wang.

Abstract

Software papers fix a scholarly context, whereas repositories continue to change. We release a version-paired static dataset linking the immutable archive associated with a Journal of Open Source Software (JOSS) article to a later immutable GitHub endpoint. The release covers 345 Python projects from JOSS articles published in 2018–2020 and contains 690 source snapshots, 9,074 declared-dependency records, 121,085 syntactic-interface records, 95,787 evidence-linked pair-delta records and 2,876 validation records. Endpoints were resolved on 14 August 2026: 327 stable releases and 18 default-branch commits. Static parsing was performed without installing, importing or executing candidate software. The tables support studies of dependency declarations, source-level interfaces, packaging, Python-version metadata, project structure, CI configuration and parser missingness. Syntactic differences are not runtime compatibility, declarations are not resolved environments, and the bounded JOSS/Python cohort is not a population-prevalence or software-quality estimate.

Background & Summary

Research software is part of the scholarly record, but a software article and its linked repository provide different temporal anchors. The article fixes a citable publication context, whereas the repository may be renamed, archived or modified after publication. A reuser who wants to study how a research tool was declared, exposed or documented therefore needs a source snapshot tied to publication and a later snapshot whose identity is immutable. Persistent software identifiers and citation practices motivate this provenance boundary, while FAIR principles require machine-readable metadata and stable access paths [1, 2].

Several public resources address complementary parts of this problem. OpenDORS links more than 134,000 open research-software projects to publications and supplies repository-level metadata [3]; SemRepo connects research repositories to scholarly knowledge graphs [4]; and the Software Heritage Graph provides broad source-history infrastructure [5]. Other datasets describe dependency networks, workflow histories, research-code execution, static-analysis reports or source-derived code-smell records [6, 7, 8, 9, 10]. These resources establish the value of reusable software traces, but their released units do not provide the same project-level object: a publication-linked immutable archive paired with a later immutable endpoint and accompanied by evidence-bearing field deltas across dependency, interface and project-structure families.

The contribution is therefore a provenance-first data object rather than a new evolution model. Each resolved project is represented by two immutable snapshots, linked through stable identifiers and accompanied by normalized source observations, row-level evidence paths, explicit comparison status and validation events. This arrangement removes a reconstruction burden for users who need to compare a published software state with a later state without silently replacing missing or partial observations. The release is designed to be joined, filtered and re-audited; it is not presented as a representative sample, a runtime benchmark or a causal study of software change.

We constructed the paired resource from the frozen Crossref frame of JOSS articles with publication dates from 1 January 2018 through 31 December 2020. Each retained project is linked to an immutable publication archive associated with the article and to the latest non-draft, non-prerelease GitHub release available on 14 August 2026. The paired cohort contains 341 Zenodo and 4 Figshare publication snapshots. When no qualifying release was available, the endpoint is the default-branch head resolved to a full commit SHA. The static pipeline records archive identity and checksums, parser versions, relative source evidence, missingness and validation events. It never installs, imports or executes third-party code.

1 Methods

1.1 Study design and unit of observation

The resource is organized around a project-level pair. A `project_id` is derived from the normalized JOSS DOI, a `pair_id` identifies the two snapshots, and each child row retains its parent `snapshot_id` or `pair_id`. A resolved project therefore has exactly one publication snapshot and one endpoint snapshot. Unresolved endpoint projects remain in the selection and endpoint ledgers but produce no fabricated snapshot or delta rows. This design makes provenance and missingness part of the data object rather than post hoc exclusions.

1.2 Frame construction and deterministic screening

The source frame was acquired from the Crossref journal works endpoint for ISSN 2475-9066 and the date interval above. The frozen frame contains 940 records. DOIs were normalized to lowercase and assigned a deterministic SHA-256 rank using the frozen screening salt. The rank controlled processing order only; it was not a sampling weight or eligibility feature. Every frame record was retained in a screening ledger with its status and reason code.

Metadata screening required a declared GitHub repository that was not identified as a fork, exposed a default branch, contained root-level Python packaging evidence and reported a machine-confirmed licence identifier. Of the 940 records, 396 passed these checks, 537 were excluded with explicit reason codes and 7 remained unresolved after official-source checks. The subsequent publication-archive audit retained 372 archive-eligible projects, excluded 20 and left 4 unresolved. Endpoint resolution was attempted for all 372 archive-eligible projects; 345 resolved and 27 remained unresolved. The 345 resolved projects form the paired extraction cohort. Explicit frame, eligibility and attrition records make the cohort boundary available for repository-mining analyses [11].

1.3 Scholarly linkage and publication snapshots

The workflow retained the JOSS DOI, article metadata, article URL and declared repository URL. Repository redirects and renames were represented by both input and canonical URLs. Publication snapshots were selected from immutable software-archive records linked to the articles. The final paired cohort uses 341 Zenodo records and 4 Figshare records. When multiple files were present, a source tar archive was preferred, then a source ZIP; built wheels were lower priority. The selected archive DOI, file key, source URL, upstream checksum, locally computed SHA-256, byte size and parser version were recorded in `snapshot_records.csv`. Publication archives were checked against their provider-reported MD5 values as well as the local SHA-256.

1.4 Endpoint policy and immutable acquisition

Endpoint selection followed a frozen release-or-commit policy. The resolver selected the latest GitHub release that was neither draft nor prerelease. If no such release existed, it selected the default-branch head. The selected reference was resolved to a 40-character commit SHA, and the source archive was addressed by that SHA rather than by a mutable branch or tag URL. Among the 345 resolved endpoints, 327 were releases and 18 were default-branch heads. Eighty-three resolutions used the GitHub REST API and 262 used official GitHub web surfaces because the unauthenticated API was rate limited. The resolution method, evidence URLs, reference, commit and archive checksum are retained in the endpoint ledger and snapshot table. The 27 unresolved projects retain their official-source failure states.

1.5 Bounded static archive access

ZIP and TAR members were read with Python archive readers without extraction to runnable locations. Absolute paths, drive-prefixed paths and path-traversal members were flagged; symbolic and hard links were counted but not followed; common wrapper directories and platform-generated `__MACOSX` members were removed. The archive reader enforced limits of 100,000 members, 2 MiB per selected text member and 128 MiB of selected text per archive. Candidate modules and build scripts were parsed as data. In particular, `setup.py` was inspected syntactically and never executed.

1.6 Dependency, interface and structure extraction

Declared direct dependencies were parsed from PEP 621 and Poetry sections in `pyproject.toml`, PEP 517 build requirements, `setup.cfg`, statically readable arguments to `setup()`, requirements files and conda environment files. Python dependency extraction depends on both the observable source evidence and the

extraction method [12]; each observation therefore preserves its source file, parser kind, dependency group, normalized name, raw declaration, specifier, marker, extras and parse status. Overlapping declaration files are retained as separate evidence; the table is not a dependency solver output.

The interface parser selected Python source files using the documented source-root and exclusion rules. It recorded non-private top-level functions and classes, methods and constructors, normalized syntactic signatures, decorators, inferred module paths and one-based source locations. Literal `__all__` declarations were preserved as export evidence when available; otherwise the record uses the `lexical_nonprivate` heuristic. Tests, examples, documentation, benchmarks, vendored directories and build directories were excluded. The result is a reproducible syntactic surface, not a runtime export or compatibility contract.

Structural extraction records packaging and build files, declared Python constraints, source-file and source-line counts, test-file counts, CI providers and workflow files, licence and citation metadata, CodeMeta and Zenodo files, changelog, contribution guide, code of conduct and security-policy presence. Presence is a binary observation of a file or configuration path and does not imply content quality or compliance.

1.7 Pairwise comparison and descriptive summaries

Dependency, interface and structure observations were normalized into family-specific comparison keys. For each key, `pair_delta_records.csv` stores the publication value, endpoint value, relative evidence and one of `added`, `removed`, `changed`, `unchanged` or `unavailable`. `comparison_status` distinguishes comparable, partial-parse and unavailable records. Aggregated counts were regenerated from these rows. Descriptive summaries use the project as the aggregation unit, report no significance tests and make no causal inference. For six scalar metrics, endpoint-minus-publication medians and 95% percentile intervals were obtained from 2,000 nonparametric project-level bootstrap resamples with seed 20260814.

1.8 Independent semantic audit

To test semantic agreement independently of the production extractor, we selected 24 projects by deterministic strata: 6 with a partial family comparison, 6 with large source trees, 5 with high API/dependency churn, 6 default-branch endpoints and 1 complete low-complexity reference. A separate audit implementation that does not import the production extractor located all 48 cached snapshots by archive size and SHA-256, reselected source members, parsed Python files with the AST module, and independently parsed structured dependency declarations. Exact agreement was evaluated for symbol keys and API kind, file, line, signature, decorator and export fields; for dependency rows and their normalized fields; and for audited structure fields. The audit reproduced 22,065 API symbol keys, 658 dependency rows and every audited structure field. These are exact-agreement results for the stratified sample, not estimates of population parser accuracy.

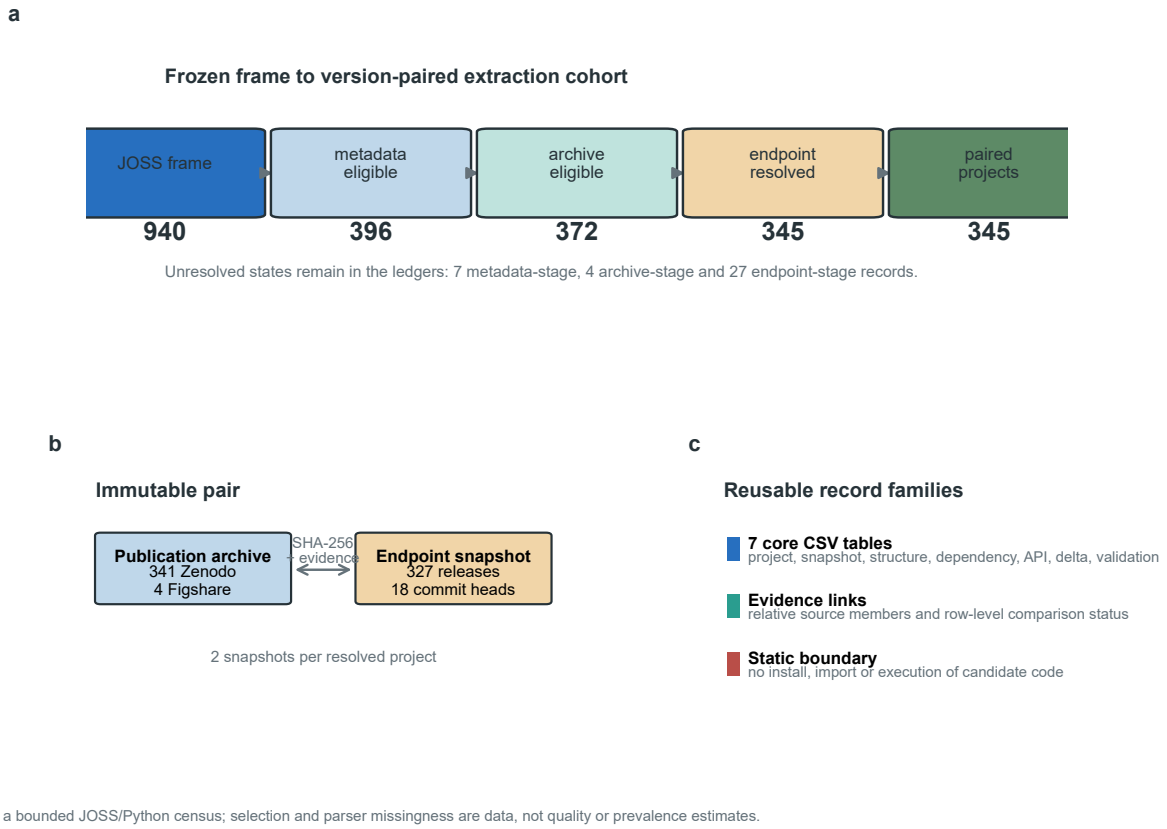


Figure 1: **Selection and version-pair data object.** (a) Deterministic attrition from the 940-record JOSS frame to 345 resolved projects. Unresolved states remain in the ledgers rather than being folded into a retained denominator. (b) Each resolved project links one immutable publication archive to one endpoint release or commit snapshot; the pair carries 690 snapshots in total. (c) The release packages seven core CSV tables, evidence links and explicit static-analysis boundaries. Source data and the generation script are provided with the figure.

2 Data Records

The v0.2.2 release contains seven UTF-8 CSV core tables, JSON manifests, schemas, controlled vocabularies, selection and endpoint ledgers, validation reports, summary tables and figure source data. The exact package is available for anonymous download from the public GitHub release. Empty strings mean that an upstream value was unavailable or not applicable; they are not imputed zeros or false values. JSON-valued cells use sorted keys and no insignificant whitespace. The core table contract is summarized in Table 1; the accompanying manifests bind row counts, schema versions and table hashes.

Table 1: Core CSV tables and their reusable units.

File	Unit	Primary reuse or join key
`project_records.csv`	project pair	`project_id`, JOSS DOI, selection and endpoint ledger
`snapshot_records.csv`	immutable snapshot	`snapshot_id`, role, source reference, SHA-256
`structure_records.csv`	snapshot summary	packaging, source/test counts, CI, governance and completeness
`dependency_records.csv`	declaration evidence	source file, parser kind, group, normalized name and specifier
`api_symbol_records.csv`	syntactic symbol	module, kind, qualified name, signature and location
`pair_delta_records.csv`	paired field comparison	`pair_id`, family, feature key, delta type and evidence paths
`validation_records.csv`	validation event	snapshot, check name, status, evidence and detail

The accompanying `extraction_manifest.json` records schema and parser versions, row counts and table hashes. The endpoint-resolution manifest records the endpoint date, policy outcomes, methods and input hashes. Summary tables provide selection attrition, endpoint types, family-level project summaries, parsing completeness, paired scalar changes, project metrics and large-project sensitivity. Figure source-data CSV files are regenerated from the validated JSON and summary tables. Raw third-party source archives are not redistributed; their archive DOI or immutable commit and checksum are retained so that users can reacquire them under the original terms.

3 Data Overview

The 345 paired projects yield 690 snapshots, 9,074 dependency observations, 121,085 syntactic interface observations, 95,787 pair-delta rows and 2,876 validation rows. Project-level net medians are modest for several scalar fields: source lines increase by 193 (95% descriptive percentile interval 87–370) and syntactic API symbols by 3 (interval 1–7), whereas the median changes in source-file count, declared dependency-record count, test-file count and CI workflow-file count are zero. Figure 2 resolves this netting effect by showing the fraction of comparable rows that were added, removed or changed within each project-family pair. At least one observable change occurs in 211/317 complete API pairs (66.6%), 133/238 complete dependency pairs (55.9%) and 243/345 structure pairs (70.4%). Binary structural fields provide a second compact view: 108 projects add a root-level `pyproject.toml`, 141 add a GitHub Actions workflow and 51 add `CITATION.cff`, whereas 123 remove Travis CI and 3 add it. Figure 3 reports both directions and retains projects that are absent at both endpoints; these counts describe file or configuration presence only.

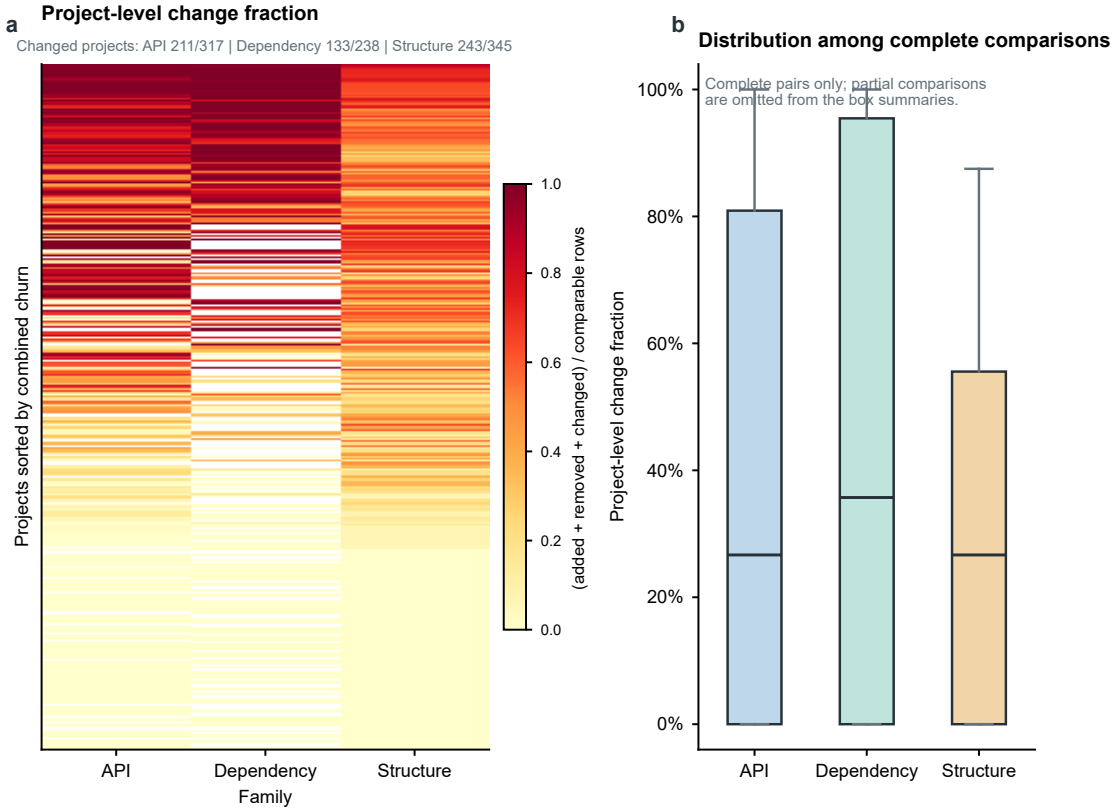


Figure 2: **Project-level static change landscape.** (a) Each row is a project and each column is a family; color encodes the fraction of comparable rows that are added, removed or changed. White cells denote partial or unavailable family comparisons. (b) Box summaries use complete project-family comparisons only. The figure exposes within-project churn that can be hidden by a net median and does not measure runtime compatibility, quality or breaking-change status. Source data are provided with the figure.

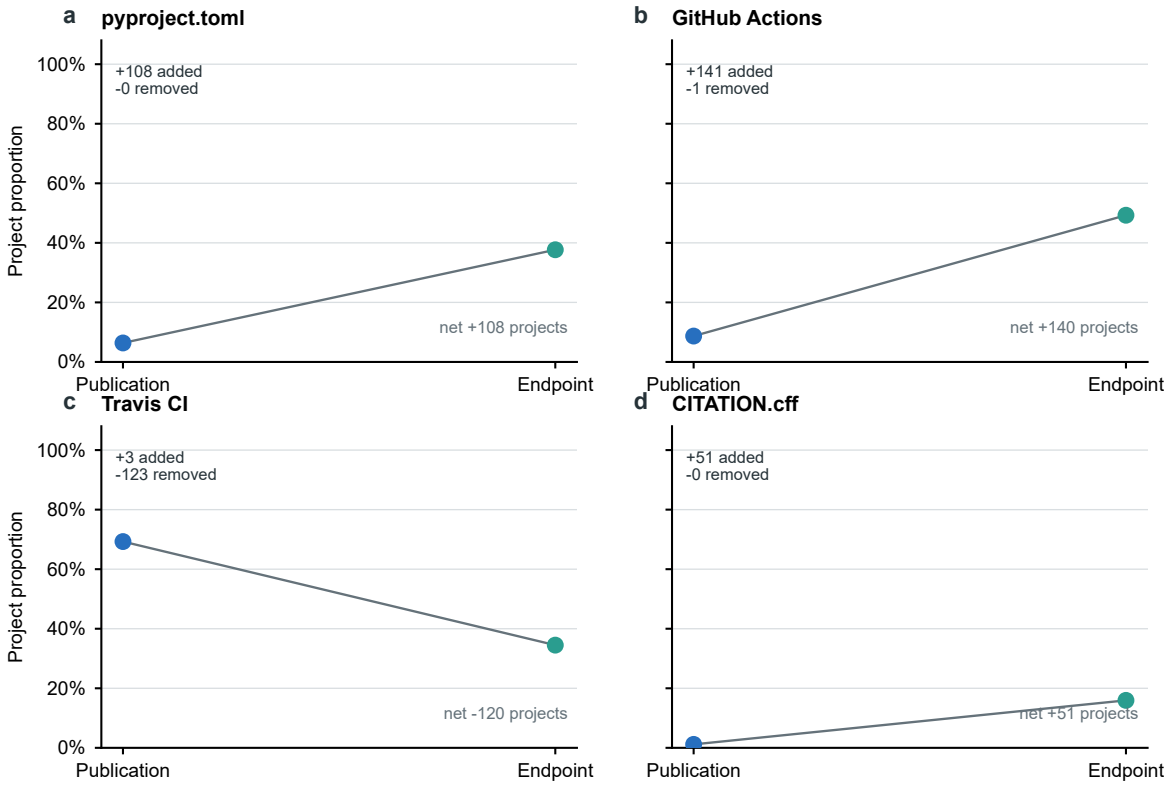


Figure 3: **Binary metadata and CI transitions.** Each slope joins the project proportion with a field present in the publication snapshot to the proportion with that field present in the endpoint snapshot. Labels report added and removed projects. The fields are `pyproject.toml`, GitHub Actions workflow files, Travis CI configuration and `CITATION.cff`; presence does not imply content quality, workflow success or citation completeness. Source data are provided with the figure.

4 Technical Validation

4.1 Archive identity and safety

All 690 acquired snapshots passed local SHA-256 and archive-member path checks. Publication archives also passed their repository-provided MD5 checks. Endpoint archives are addressed by resolved commit SHAs, and the endpoint ledger preserves the resolution method and evidence URL. The archive validator, schema validator and controlled-vocabulary checks reported no failures. The static pipeline recorded `third_party_code_executed=false` in both the extraction manifest and validation records.

4.2 Parsing completeness and failure retention

Structure records are complete for all 345 projects at both snapshots. For API records, 322 publication snapshots, 320 endpoint snapshots and 317 paired projects have complete parsing status. Dependency parsing is complete for 250 publication snapshots, 265 endpoint snapshots and 238 paired projects. The remaining states are explicit in the structure and delta tables; no missing value was replaced by zero. The validation table retains 157 expected `python_ast_parse` errors and 8 `api_source_read` errors, together with 179 metadata-parse warnings and 1,152 AST warnings. These events are parser limitations to be stratified or excluded by downstream users, not evidence that the source code was executed.

4.3 Independent semantic agreement

The independent 24-project audit located all 48 sampled archives and exactly reproduced the released API symbol keys and audited fields (22,065 keys), dependency rows (658 rows) and structure fields. The audit was deliberately stratified to include partial parsing, large source trees, high-churn projects, default-branch

endpoints and a complete reference. It confirms that the released records can be reconstructed from the archived bytes under the stated static rules; it does not establish parser precision or recall for the full 345-project cohort. Figure 4 places this semantic check beside completeness and retained parser events.

4.4 Referential integrity and deterministic rerun

The strict census validator reported PASS for primary-key uniqueness, two-snapshot pairing, referential integrity, controlled vocabularies, archive hashes, path safety, evidence-path relativity and the no-execution assertion. A clean rerun from cached immutable inputs produced byte-identical canonical tables. The released manifests bind the input and output SHA-256 values, enabling a reuser to verify that the seven core tables and summary products correspond to the stated parser version.

As a separate provenance spot-check, a deterministic 20-project sample was reconstructed from the cached archives without importing the extraction implementation. All 40 sampled snapshot archives matched their recorded sizes and SHA-256 values, and all 38 sampled API or dependency evidence paths were found in the corresponding normalized archive member sets. This earlier audit is a bounded reconstruction check rather than a population accuracy estimate; its ledger is released as `independent_reconstruction_audit.csv` with a machine-readable summary.

4.5 Selection and endpoint accounting

The selection ledger accounts for all 940 frame records. The attrition path is retained as data rather than collapsed into a single denominator: 940 screened, 396 metadata-eligible, 372 publication-archive eligible, 345 endpoint-resolved and 345 paired-extracted. The 7 metadata-stage unresolved records, 4 archive-stage unresolved records and 27 endpoint-unresolved records remain visible with reason codes. Consequently, the 345-project paired summaries use the resolved-pair denominator, whereas endpoint-resolution proportions use the 372 archive-eligible denominator when unresolved endpoints are shown.

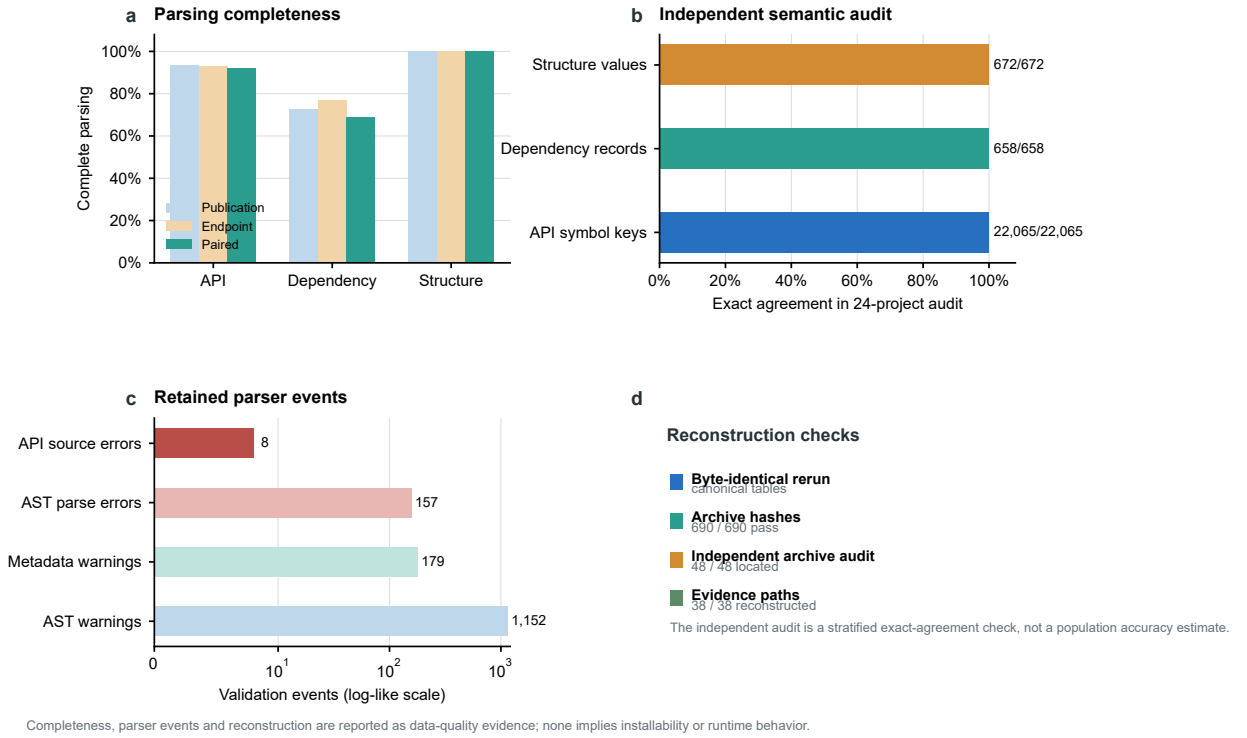


Figure 4: **Completeness and validation evidence.** (a) Complete parsing proportions for publication, endpoint and paired snapshots. (b) Exact agreement in the independent 24-project semantic audit. (c) Retained AST, metadata and source-read events, including expected static parse limitations. (d) Independent reconstruction and archive checks. These panels document data-generation and validation boundaries; none implies installability, runtime behavior or population-level parser accuracy. Source data are provided with the figure.

5 Usage Notes

Users can join `project_records.csv` to `snapshot_records.csv` through `project_id`, attach child observations through `snapshot_id`, and join cross-snapshot comparisons through `pair_id`. Analyses of dependency or interface changes should filter or stratify by `comparison_status`; partial and unavailable rows must not be silently treated as complete. Empty cells denote unavailable or inapplicable upstream values, while a recorded zero is a measured zero.

The principal reuse cases are (i) attrition-aware studies of publication-to-repository linkage, (ii) comparison of declared dependency and Python-version metadata parsers, (iii) benchmarking of source-level interface extraction, (iv) analyses of test and CI configuration co-occurring with other static changes, and (v) methods studies of missingness, parser warnings and deterministic data reconstruction. These uses can be implemented from the released tables without rebuilding the complete archive-pairing pipeline. The project and snapshot identifiers also make it possible to join the records to later registry or source-history resources while preserving the original provenance boundary.

The syntactic interface heuristic excludes private names and selected non-source directories but cannot reproduce Python import semantics, dynamic exports, conditional imports or generated interfaces. Syntactic differences require a language- and compatibility-specific interpretation and must not be treated automatically as breaking changes [13, 14, 15]. Dependency rows describe declarations and do not encode solver results, installed versions or transitive closure; execution-oriented build reproducibility and dependency-error repair remain outside the dataset [16]. Source-line counts, extracted-row volumes and metadata-file presence are not quality, maintenance, sustainability or impact scores. The external scope is limited to JOSS articles in the 2018–2020 frame, repositories meeting the stated GitHub/Python/licence rules and endpoint state resolved on 14 August 2026.

Data Availability

The derived dataset supporting this Data Descriptor is available for anonymous download in the public v0.2.2 release. The package contains the seven core tables, schemas, manifests, controlled vocabularies, selection and endpoint ledgers, validation products, figure source data and checksums under the Creative Commons Attribution 4.0 International licence. Raw third-party source archives are not redistributed; each source snapshot is identified by a public archive DOI or immutable Git commit together with a local SHA-256 value. A DOI-bearing formal data-repository record will replace or supplement this first-round access route before the second round of review.

Code Availability

The custom code used to construct the frame, resolve endpoints, inspect archives, extract static records, construct deltas, run the independent audits, generate the four figures and validate the release is available from the public GitHub repository and its versioned v0.2.2 release under the MIT License. The release includes the pinned reference environment, figure source-data exports and the exact manuscript package. A DOI-minted archival copy will be linked before the second round of review.

References

[1] Laura Soito and Lorraine J. Hwang. Citations for software: Providing identification, access and recognition for research software. *International Journal of Digital Curation*, 11(2):48–63, 2017.

[2] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, Jildau Bouwman, Anthony J. Brookes, Tim Clark, Merce Crosas, Ingrid Dillo, Olivier Dumon, Scott Edmunds, Chris T. Evelo, Richard Finkers, Alejandra Gonzalez-Beltran, Alasdair J. G. Gray, Paul Groth, Carole Goble, Jeffrey S. Grethe, Jaap Heringa, Peter A. C. t Hoen, Rob Hooft, Tobias Kuhn, Ruben Kok, Joost Kok, Scott J. Lusher, Maryann E. Martone, Albert Mons, Abel L. Packer, Bengt Persson, Philippe Rocca-Serra, Marco Roos, Rene van Schaik, Susanna-Assunta Sansone, Erik Schultes, Thierry Sengstag, Ted Slater, George Strawn, Morris A. Swertz, Mark Thompson, Johan van der Lei, Erik van Mulligen, Jan Velterop, Andra Waagmeester, Peter Wittenburg, Katherine Wolstencroft, Jun Zhao, and Barend Mons. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3(1), 2016.

[3] Stephan Druskat and Lars Grunske. Opendors: A dataset of openly referenced open research software, 2025. Related dataset: https://doi.org/10.5281/zenodo.17560612.

[4] Abdul Rafay, Yuni Susanti, David Lamprecht, and Michael Farber. Semrepo: A knowledge graph for research software and its scholarly ecosystem, 2026. Related dataset: https://doi.org/10.5281/zenodo.20084784.

[5] Antoine Pietri, Diomidis Spinellis, and Stefano Zacchiroli. The software heritage graph dataset: Public software development under one roof. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories*, pages 138–142, 2019.

[6] William Schueller, Johannes Wachs, Vito D. P. Servedio, Stefan Thurner, and Vittorio Loreto. Evolving collaboration, dependencies, and use in the rust open source software ecosystem. *Scientific Data*, 9(1), 2022.

[7] Guillaume Cardoen, Tom Mens, and Alexandre Decan. A dataset of github actions workflow histories. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 677–681, 2024.

[8] Ana Trisovic, Matthew K. Lau, Thomas Pasquier, and Merce Crosas. A large-scale study on research code quality and execution. *Scientific Data*, 9(1), 2022.

[9] David Koszo, Tamas Aladics, Rudolf Ferenc, and Peter Hegedus. A large-scale collection of (non-)actionable static code analysis reports. *Scientific Data*, 12(1), 2025.

[10] Nawaf Alomari, Amal Alazba, Hamoud Aljamaan, and Mohammad Alshayeb. Smellycode++: Multi-label dataset for code smell detection. *Scientific Data*, 12(1), 2025.

[11] Mikel Robredo, Nyyti Saarimaki, Valentina Lenarduzzi, Sira Vegas, and Davide Taibi. Cohort studies for mining software repositories. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 569–570, 2024.

[12] Wuxia Jin, Shuo Xu, Dawei Chen, Jiajun He, Dinghong Zhong, Ming Fan, Hongxu Chen, Huijia Zhang, and Ting Liu. Pyanalyzer: An effective and practical approach for dependency extraction from python code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12, 2024.

[13] Dhanushka Jayasuriya, Samuel Ou, Saakshi Hegde, Valerio Terragni, Jens Dietrich, and Kelly Blincoe. An extended study of syntactic breaking changes in the wild. *Empirical Software Engineering*, 30(2), 2025.

[14] Tarek Mahmud, Meiru Che, and Guowei Yang. Detecting android api compatibility issues with api differences. *IEEE Transactions on Software Engineering*, 49(7):3857–3871, 2023.

[15] Chris Bogart, Christian Kastner, James Herbsleb, and Ferdian Thung. When and how to make breaking changes. *ACM Transactions on Software Engineering and Methodology*, 30(4):1–56, 2021.

[16] Suchita Mukherjee, Abigail Almanza, and Cindy Rubio-Gonzalez. Fixing dependency errors for python build reproducibility. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 439–451, 2021.

Author Contributions

Author contributions are provided in the journal submission system.

Competing Interests

The authors declare no competing interests.

Acknowledgements

The authors acknowledge the public Crossref, JOSS, Zenodo, Figshare and GitHub services used to construct the frame and immutable source links. AI-assisted tools were used for language drafting and code-oriented quality checks; all data-generation decisions, source interpretation and final claims remain the authors' responsibility.

Funding

This work received no external funding.

Ethics Statement

This work uses public scholarly records and public repository metadata and does not involve human participants, private repositories or interaction with contributors. The release plan excludes raw third-party archives unless redistribution rights are established and follows the access and licence conditions of the original providers.